Documentation for `RandomTilings 1.2.0`

Max van Horssen & Lennart Hübner

Department of Mathematics, KU Leuven,
Celestijnenlaan 200 B bus 2400, 3001 Leuven, Belgium
max.vanhorssen@kuleuven.be, lennart.huebner@kuleuven.be

October 27, 2026

Abstract

This document provides the documentation for the Python package `RandomTilings 1.2.0`, which is a Python adaptation of the Matlab program `MatlabTilings` by Christophe Charlier. The underlying algorithm is the domino shuffling algorithm as described in arXiv:0111034. We are grateful to Christophe Charlier for allowing us to make this package publicly available.

Contents

0 About the Python package `RandomTilings` 1

1 Random tilings of the Aztec diamond 2
1.1 `RT.Aztec` . . . 2
1.2 `A.shuffle` . . . 2
1.3 `A.plot` . . . 2
1.4 `A.plot_dimers` . . . 3
1.5 `A.plot_paths` . . . 3
1.6 `A.get_log_partition_function` . . . 3
1.7 Basic examples . . . 3
1.8 An advanced example: Crystallization of the Aztec diamond . . . 5

2 Random tilings of the hexagon 7
2.1 `RT.Hexagon` . . . 7
2.2 `H.shuffle` . . . 7
2.3 `H.plot` . . . 7
2.4 `H.plot_dimers` . . . 8
2.5 `H.plot_paths` . . . 8
2.6 `H.get_log_partition_function` . . . 8
2.7 Basic examples . . . 8

0 About the Python package `RandomTilings`

This project provides the Python package `RandomTilings`, which makes it possible to generate random tilings of the Aztec diamond and the hexagon for doubly periodic weightings. This package is a Python adaptation of the MATLAB program `MatlabTilings` by Christophe Charlier, which is based on the domino shuffling algorithm as described in arXiv:0111034. The original MATLAB implementation can be found on his webpage. We are grateful to Christophe Charlier for allowing us to make this package publicly available.

The package `RandomTilings` requires to install the following packages: `NumPy`, `Matplotlib`, `ipympl`, `ipywidgets`, `Numba`, and `tqdm`. Make sure that these libraries are installed beforehand. Once the installation is successful, you only have to copy the folder `RandomTilings` in the same folder as your `Python script` or `Jupyter notebook`, and you can import the routines by simply calling:

```
import RandomTilings as RT
```

Note that by importing all necessary subroutines will be compiled by `Numba.njit`, therefore the first time importing this library might take some time.

1 Random tilings of the Aztec diamond

1.1 RT.Aztec

To sample a random tiling of the Aztec diamond, one first has to construct an 'Aztec' object.

```
A = RT.Aztec(n)
```

1. `n` is the size of the Aztec diamond.

There are two *optional arguments*:

```
A = RT.Aztec(n,w,gap)
```

2. `w` is the weighting on the edge graph, which can be a matrix of any size. We adopt the convention that the weighting is assigned on the bottom left corner of the Aztec diamond as shown in Figure 1, and is then periodically extended to the full Aztec diamond. By *default*, the uniform weighting is used, i.e., `w = [[1]]`.

3. `gap` must be of the form `[[t1,x1,y1],[t2,x2,y2],...,[tm,xm,ym]]`, which means that at each time `tj`, there is a vertical gap from `xj` to `yj`. The points must satisfy `tj` $\in \{0, 1, \ldots, 2n\}$ and `xj`,`yj` $\in \mathbb{Z}$. It is allowed to take `xj = yj`, which represents a single point `xj`.

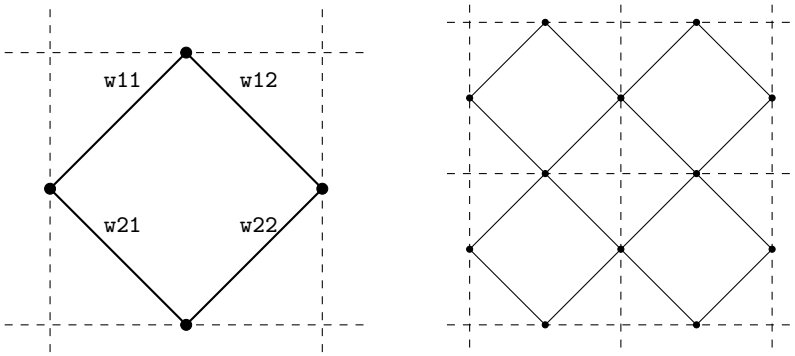


Figure 1: The weighting `w = [[w11,w12],[w21,w22]]` on the edge graph, extended to the Aztec diamond of size 2.

1.2 A.shuffle

Once the Aztec object `A` is constructed, it can be used to sample the Aztec diamond by calling `shuffle`.

```
A.shuffle()
```

This routine executes the *domino shuffle algorithm*. It can be called repeatedly without needing to reconstruct the Aztec object.

1.3 A.plot

Having sampled the Aztec diamond, one can plot it by calling `plot`.

```
A.plot()
```

There are several *optional arguments*:

1. `border_size` is the thickness of the border around the domino tiles. By *default*, `border_size = 0`, resulting in no border being drawn.

2. `boundary_size` is the thickness of the boundary around the tiling. By *default*, `boundary_size = 0`, resulting in no boundary being drawn.

3. `orientation` gives the orientation of the figure, for which two options are available: `'diamond'` and `'square'`. By *default*, `orientation = 'diamond'`.

4. `color_scheme` is the color scheme of the figure. Five built-in options are available: `'standard'`, `'alternative'` (taken from arXiv:2402.08798), `'gray'`, `'aztec gray'` (taken from arXiv:1410.2385; custom-made for the 2×2-periodic Aztec diamond), and `'tropical'` (taken from arXiv:2605.03896). Custom color schemes are specified as `[(r,g,b),(r,g,b),(r,g,b),(r,g,b)]`, where the four RGB triples correspond to the colors of the north, west, south, and east dominoes, respectively. Each color component `r`, `g`, and `b` may be given either as an integer in $\{0, \ldots, 255\}$ or as a real number in $[0, 1]$. By *default*, `color_scheme = 'standard'`.

5. `dpi` is the resolution of the figure. By *default*, `dpi = 100`.

1.4 A.plot_dimers

Having plot the Aztec diamond, one can add the corresponding dimer configuration to the existing figure by calling `plot_dimers`.

```
A.plot_dimers()
```

There are several *optional arguments*:

1. `line_size` is the thickness of the dimers in the dimer configuration. By *default*, a scaling size is implemented.

2. `dot_size` is the thickness of the dots at both ends of the dimers. By *default*, a scaling size is implemented.

3. `dot_border_size` is the thickness of border of the dots at both ends of the dimers. By *default*, a scaling size is implemented.

1.5 A.plot_paths

Having plot the Aztec diamond, one can add the corresponding non-intersecting path system to the existing figure by calling `plot_paths`.

```
A.plot_paths()
```

There are several *optional arguments*:

1. `path_size` is the thickness of the paths in the non-intersecting path system. By *default*, a scaling size is implemented.

2. `dot_size` is the thickness of the dots visualizing the particles associated with the non-intersecting path system. By *default*, a scaling size is implemented.

3. `gap_size` is the thickness of the lines visualizing the gaps. By *default*, `gap_size = 0`.

1.6 A.get_log_partition_function

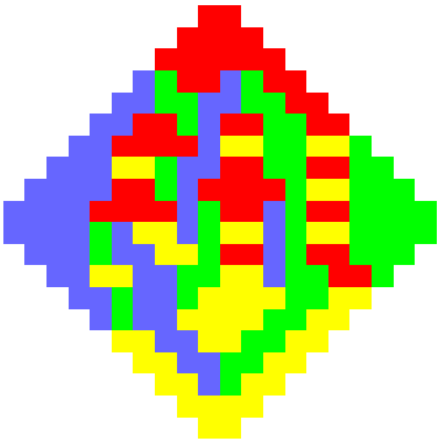
Once the Aztec object `A` is constructed, one can obtain the log partition function from it by calling `get_log_partition_function`.

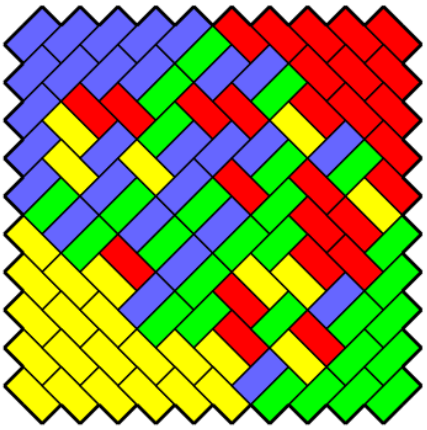
```
A.get_log_partition_function()
```

This routine returns a complex number. A non-zero imaginary part indicates that there is no tiling with the specified weighting. The real part gives the log of the partition function, if applicable.

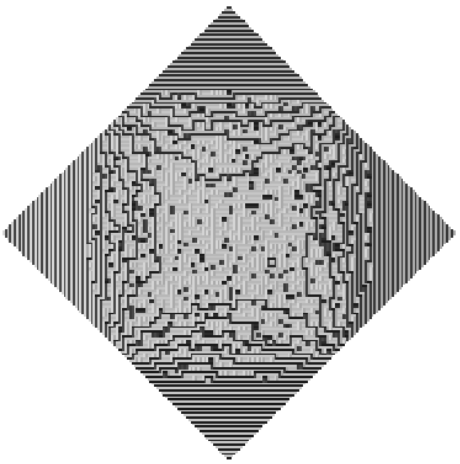
1.7 Basic examples

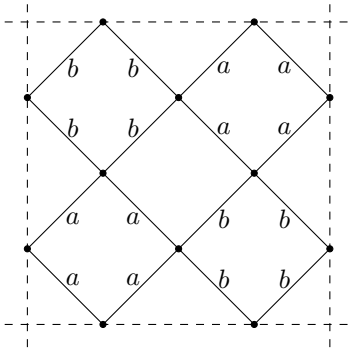
```
import RandomTilings as RT
A = RT.Aztec(10)
A.shuffle()
A.plot()
A.plot(border_size=1,boundary_size=2,orientation='square')
A.close()
```



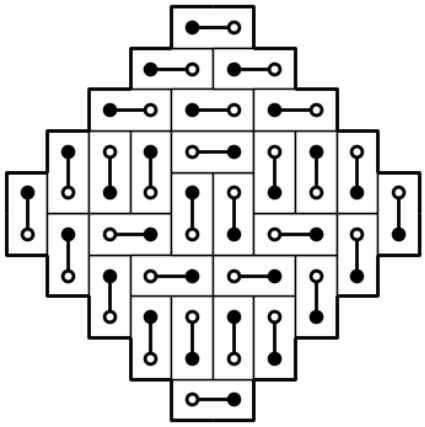


```
import RandomTilings as RT
a = 0.5
b = 1
w = [[b,b,a,a],[b,b,a,a],[a,a,b,b],[a,a,b,b]]
A = RT.Aztec(100,w)
A.shuffle()
A.plot(color_scheme='aztec gray')
A.close()
```

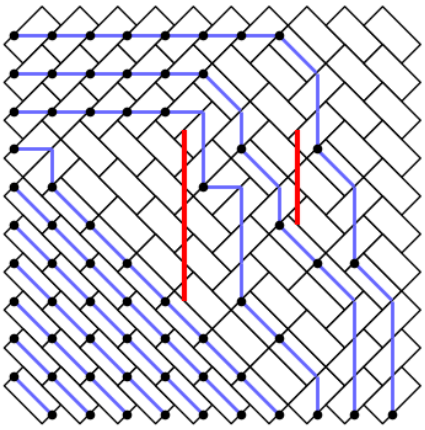





```
import RandomTilings as RT
A = RT.Aztec(5)
A.shuffle()
A.plot(border_size=1,boundary_size=2,color_scheme='white')
A.plot_dimers(line_size=2,dot_size=40,dot_border_size=2)
A.close()
```



```
import RandomTilings as RT
gap = [[9,-2,2],[15,3,5]]
A = RT.Aztec(10,gap=gap)
A.shuffle()
A.plot(border_size=1,orientation='square',color_scheme='white')
A.plot_paths(path_size=2,dot_size=20,gap_size=3)
A.close()
```



```
import RandomTilings as RT
a = 0.5
n = 5
A = RT.Aztec(n,[[a,1],[1,a]])
logPartitionFunction = A.get_log_partition_function()
A.close()
print("Do valid tilings exist?", logPartitionFunction.imag==0)

from numpy import exp
partitionFunction = exp(logPartitionFunction.real)
print(partitionFunction)

# Exact value according to Elkies, Kuperberg, Larsen, and Propp (1992)
exactPartitionFunction = (1+a**2)**(n*(n+1)/2)
print(exactPartitionFunction)
```

This example code returns "Do valid tilings exist? True", `28.421709430404064`, and `28.421709430404007`.

1.8 An advanced example: Crystallization of the Aztec diamond

This example is taken from arXiv:2605.03896, see also arXiv:2410.04187. It illustrates how one may sample weightings that contain expressions of the form $c\exp(-n\tau)$ with $c \in \mathbb{R}_{>0}$ and $n \in \mathbb{N}$ as $\tau \to \infty$.

The package handles this symbolically by treating $\exp(-n\tau)$ as a formal variable ε. Expressions of the form $c\exp(-n\tau)$ are encoded by the complex number: `c + n*1j`.

```
import RandomTilings as RT
from numpy import pi, abs, sin

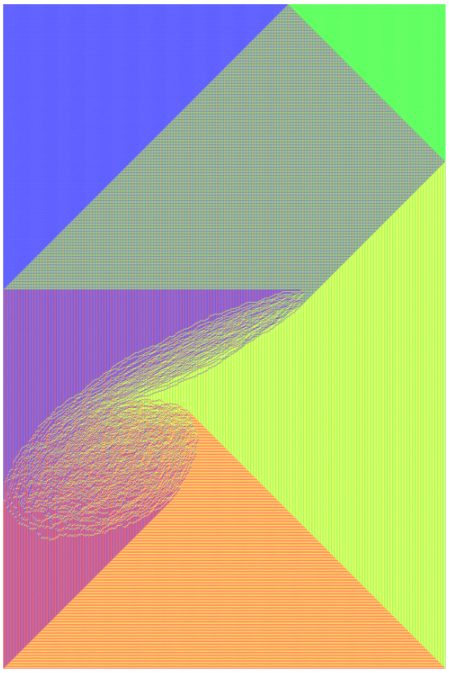
# Enable a more numerically stable implementation
RT.config.log_variant = True
# Enable an implementation optimized for the Aztec diamond with period weights
RT.config.periodic_aztec = True

def diff(theta,phi):
    return abs(sin((theta-phi)/2))

alpha = 0.
beta = pi/25
gamma = 4*pi/5
delta = 6*pi/5
epsilon = 8*pi/5

w = [[1+1j,diff(epsilon,gamma),diff(gamma,beta),diff(delta,gamma),diff(gamma,
    beta),diff(beta,gamma)],
[diff(gamma,beta),diff(beta,epsilon),1+4*1j,diff(beta,delta),1+1j,1+4*1j],
[diff(delta,gamma),diff(epsilon,delta),diff(delta,beta),1+1j,diff(delta,alpha),
    diff(alpha,delta)],
[diff(gamma,alpha),diff(alpha,epsilon),diff(beta,alpha),diff(alpha,delta),1+4*1
    j,1+1j]]

A = RT.Aztec(1000,w)
A.shuffle()
A.plot(orientation='square',color_scheme='tropical',dpi=200)
fig = A.get_figure()
ax = fig.axes[0]
ax.set_aspect(3/2)
ax.invert_xaxis()
ax.invert_yaxis()
A.close()
```



2 Random tilings of the hexagon

2.1 RT.Hexagon

To sample a random tiling of the hexagon, one first has to construct a 'Hexagon' object.

```
H = RT.Hexagon(n)
```

1. `n` is the size of the hexagon

There are five *optional arguments*:

```
H = RT.Hexagon(n,w,a,b,c,gap)
```

2. `w` is the weighting on the edge graph, which can be a matrix of any size. For a $p \times q$ periodic weighting, `w` must be a matrix of size $2p \times q$. We adopt the convention that the weighting is assigned on the bottom left corner of the hexagon as shown in Figure 2, and is then periodically extended to the full hexagon. By *default*, the uniform weighting is used, i.e., `w = [[1],[1]]`.

3.–5. `a`, `b`, and `c` are the side length multipliers, i.e., the hexagon will be of size $an \times bn \times cn$. By *default*, `a = b = c = 1`.

6. `gap` must be of the form `[[t1,x1,y1],[t2,x2,y2],...,[tm,xm,ym]]`, which means that at each time `tj`, there is a vertical gap from `xj` to `yj`. The points must satisfy `tj` $\in \{0, 1, \ldots, 2n\}$ and `xj`,`yj` $\in \frac{1}{2} + \mathbb{Z}$. It is allowed to take `xj = yj`, which represents a single point `xj`.

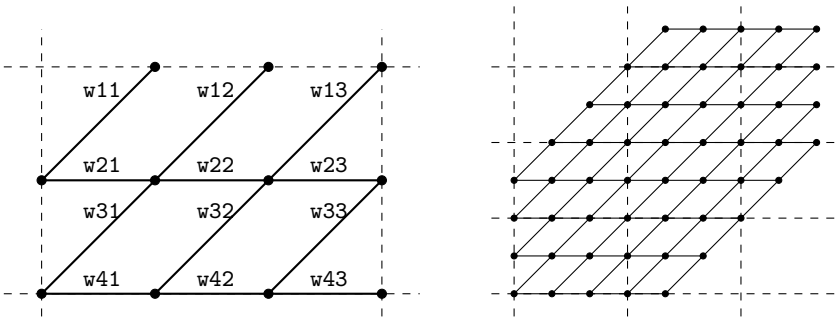


Figure 2: The weighting `w = [[w11,w12,w13],[w21,w22,w23],[w31,w32,w33],[w41,w42,w43]]` on the edge graph, extended to the hexagon of size $4 \times 4 \times 4$.

2.2 H.shuffle

Once the Hexagon object `H` is constructed, it can be used to sample the Hexagon by calling `shuffle`.

```
H.shuffle()
```

This routine executes the *domino shuffle algorithm* by first embedding the hexagon inside a larger Aztec diamond. It can be called repeatedly without needing to reconstruct the Hexagon object.

2.3 H.plot

Having sampled the Hexagon, one can plot it by calling `plot`.

```
H.plot()
```

There are several *optional arguments*:

1. `border_size` is the thickness of the border around the domino tiles. By *default*, `border_size = 0`, resulting in no border being drawn.

2. `boundary_size` is the thickness of the boundary around the tiling. By *default*, `boundary_size = 0`, resulting in no boundary being drawn.

3. `shape` gives the shape of the figure, for which two options are available: `'regular'` and `'skewed'`. By *default*, `shape = 'regular'`.

4. `color_scheme` is the color scheme of the figure. Three built-in options are available: `'standard'`, `'alternative'` (taken from arXiv:2402.08798), and `'gray'`. Custom color schemes are specified as `[(r,g,b),(r,g,b),(r,g,b)]`, where the three RGB triples correspond to the colors of the left, right, horizontal lozenges, respectively. Each color component `r`, `g`, and `b` may be given either as an integer in $\{0,\dots,255\}$ or as a real number in $[0,1]$. By *default*, `color_scheme = 'standard'`.

5. `dpi` is the resolution of the figure. By *default*, `dpi = 100`.

2.4 H.plot_dimers

Having plot the Hexagon, one can add the corresponding dimer configuration to the existing figure by calling `plot_dimers`.

```
A.plot_dimers()
```

There are several *optional arguments*:

1. `line_size` is the thickness of the dimers in the dimer configuration. By *default*, a scaling size is implemented.

2. `dot_size` is the thickness of the dots at both ends of the dimers. By *default*, a scaling size is implemented.

3. `dot_border_size` is the thickness of border of the dots at both ends of the dimers. By *default*, a scaling size is implemented.

2.5 H.plot_paths

Having plot the Hexagon, one can add the corresponding non-intersecting path system to the existing figure by calling `plot_paths`.

```
A.plot_paths()
```

There are several *optional arguments*:

1. `path_size` is the thickness of the paths in the non-intersecting path system. By *default*, a scaling size is implemented.

2. `dot_size` is the thickness of the dots visualizing the particles associated with the non-intersecting path system. By *default*, a scaling size is implemented.

3. `gap_size` is the thickness of the lines visualizing the gaps. By *default*, `gap_size = 0`.

2.6 H.get_log_partition_function

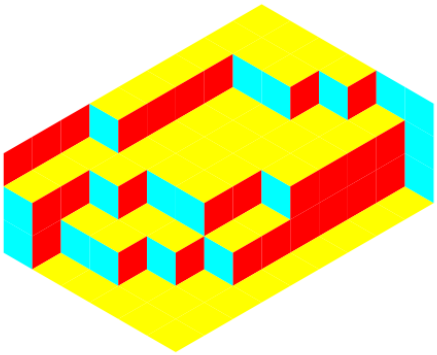
Once the Hexagon object `H` is constructed, one can obtain the log partition function from it by calling `get_log_partition_function`.

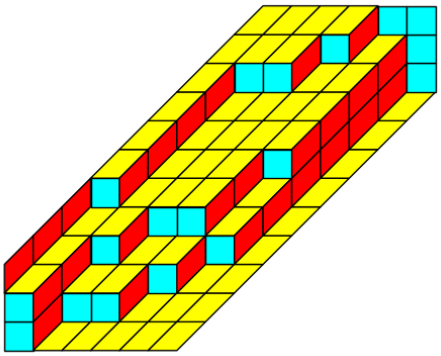
```
H.get_log_partition_function()
```

This routine returns a complex number. A non-zero imaginary part indicates that there is no tiling with the specified weighting. The real part gives the log of the partition function, if applicable.

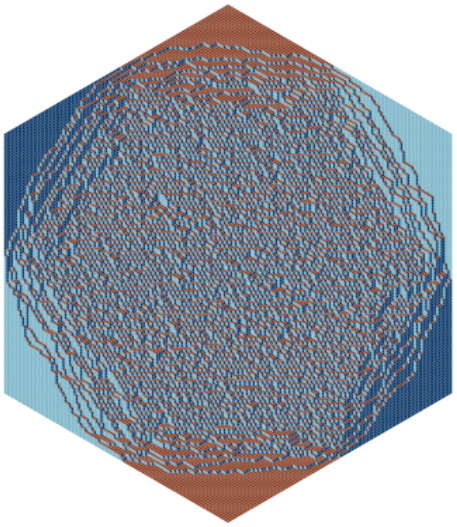
2.7 Basic examples

```
import RandomTilings as RT
H = RT.Hexagon(3,a=1,b=2,c=3)
H.shuffle()
H.plot()
H.plot(border_size=1,boundary_size=2,shape='skewed')
H.close()
```

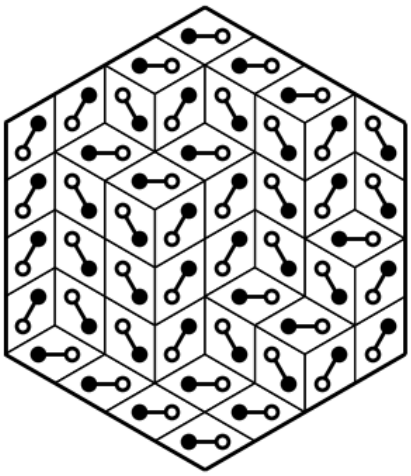




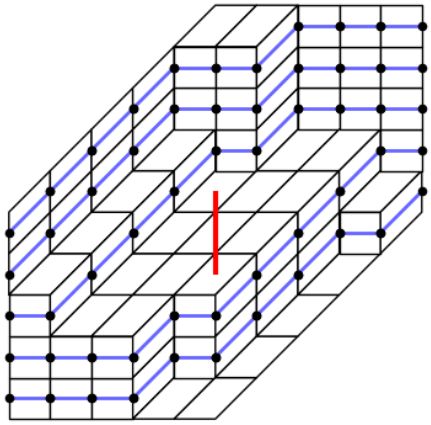
```
import RandomTilings as RT
a1 = 0.3
a2 = 0.3
w = [[1,1,1],[a2/a1,a1/a2,1],[1,1,1],[1/a2,a2,1],[1,1,1],[a1,1/a1,1]]
H = RT.Hexagon(100,w)
H.shuffle()
H.plot(color_scheme=[(5,50,100),(120,180,200),(135,60,35)])
H.close()
```



```
import RandomTilings as RT
A = RT.Hexagon(4)
A.shuffle()
A.plot(border_size=1,boundary_size=2,color_scheme='white')
A.plot_dimers(line_size=2,dot_size=40,dot_border_size=2)
H.close()
```



```
import RandomTilings as RT
gap = [[5,3.5,5.5]]
H = RT.Hexagon(5,gap=gap)
H.shuffle()
H.plot(border_size=1,shape='skewed',color_scheme='white')
H.plot_paths(path_size=2,dot_size=20,gap_size=3)
H.close()
```



```
import RandomTilings as RT
n = 2
a,b,c = 1,2,3
H = RT.Hexagon(n,a=a,b=b,c=c)
logPartitionFunction = H.get_log_partition_function()
H.close()
print("Do valid tilings exist?", logPartitionFunction.imag==0)

from numpy import exp
partitionFunction = exp(logPartitionFunction.real)
print(partitionFunction)

# MacMahon's formula
A,B,C = a*n,b*n,c*n
exactPartitionFunction = 1
for i in range(1,A+1):
    for j in range(1,B+1):
        for k in range(1,C+1):
            exactPartitionFunction *= (i+j+k-1)/(i+j+k-2)
print(exactPartitionFunction)
```

This example code returns "Do valid tilings exist? True", `13859.99999999998`, and `13860.0`.